

Recursion: Overview and Objectives

What recursion is, and what you should be able to do by the end of the unit

1. Overview

Recursion is a way of solving a problem by having a method call itself on a smaller version of the same problem. It is the first topic in the course on purpose: almost everything later, including trees, sorting, backtracking, and divide and conquer, leans on the habit of mind you build here. If you can look at a problem and see a smaller copy of it hiding inside, the rest of the semester gets a lot easier.

We work in **two languages at once**. Computer Science students write **C/C++**; CIS students write **Java**. The recursion *idea* is identical in both: a method that calls itself, a base case, and a return (backtrack) path. So most examples appear side by side. Where the languages genuinely differ (how arguments are passed, arrays versus `std::vector`, the risk of stack overflow), we call it out.

You should already be comfortable with one-dimensional arrays. A few examples reach back to arrays, and a couple look ahead to two-dimensional grids so you can see that recursion is not just for arithmetic.

Class time is mostly **live**: hand-drawn traces and coded examples we work through together, with you responding and predicting as we go. Read ahead, try the examples yourself, and come ready to think out loud. The slides are a skeleton; the drawings and code are the muscle.

2. Learning objectives

By the end of the unit you should be able to:

Remember and understand

- Recall the definition of recursion and state the purpose it serves.
- Discuss the features of a recursive solution: **base case**, **recursive step**, and the **return (back-track)** path.
- Recognize when a piece of code is recursive, including indirect (mutual) recursion.

Apply

- Respond to a problem by writing a correct recursive function or method in your language.
- Carry out standard recursive algorithms (for example, factorial, sum of a range, binary search, Towers of Hanoi).
- Trace a recursive process by hand using a **box trace** or **recursion tree**, and relate it to the **call-stack** concept.

Analyze

- Classify a recursive problem as **linear** versus **tree** recursion.
- Diagram a recursive solution as a **box trace** and as a **recursion tree**.
- Predict when recursion is an appropriate strategy, and when a loop is the better choice.
- Identify a missing or unreachable base case, and explain why it causes infinite recursion or stack overflow.

Evaluate and create

- Select the most efficient strategy for a given recursive problem (for example, recognizing recomputation in naive Fibonacci).
- Generate a complete recursive coded solution to a new problem from a plain-English description.
- Convert a simple recursive solution to an iterative one, and the reverse, and say what you gained or lost.